Solitary waves in shallow water: comparisons with the one-soliton solution of the KdV equation

Cristian Papanaga and Luis León Vintró*
Advanced Laboratories, University College Dublin
cristian.papanaga@ucdconnect.ie
(Dated: October 27, 2022)

Following a brief exploration of the theory of solitons, we demonstrate that 7 primary predictions of the solitary solution to the KdV equation hold for a set of 67 nonlinear waves in water, provided their Ursell number is sufficiently low. In particular, we derive the relationship between amplitude and speed for a set of 38 solitons, derive the phase shift of solitary water waves in catch-up collisions, and observe the number of waves which evolve from a given set of initial conditions, and demonstrate that these are in agreement with soliton theory within stated uncertainties.

I. INTRODUCTION

The solitary wave, at its inception, was associated primarily with the theory of water waves; take, for example, Scott Russell's often-quoted 1845 observation of long waves in shallow water, which seemed to propagate with invariant shape and speed [1]. Today, solitary waves are primarily investigated as exact solutions of certain nonlinear partial differential equations known to be characterised by two properties [2]:

- they are of permanent shape and speed,
- and they are localized within a region, such that they decay to a constant at infinity.

The 1955 numerical work of Fermi, Pasta, Ulam, and Tsingou [3], followed by the 1965 simulations of Zabusky and Kruskal [4], established a third property which only some solitary waves, called solitons, exhibit:

- they retain their identities in nonlinear interactions with other solitons, experiencing, at most, a phase shift.

In this work we seek to use the modern understanding of solitary waves, and soliton theory, to investigate the degree to which long waves in shallow water, as observed by Scott-Russell, can be accurately modelled by the one-soliton solution to the Korteweg-De Vries (KdV) equation.

II. KORTEWEG-DE VRIES EQUATION

The Korteweg-De Vries (KdV) equation, eponymously named for its initial publishers, is a weakly nonlinear partial differential equation developed to describe water waves moving in shallow water (or, equivalently, water waves of sufficiently long wavelength).

A complete derivation of the KdV equation lies beyond the extent of our work (for such a derivation from both the Laplace and Euler equations see Johnson [5]), but we include here an outline of the physical assumptions and mathematical methods which lead to it, so that our work is sufficiently motivated and analytically sound; consider, for example, the definition of a 'long' wave, or a 'small' amplitude. Only by a rigorous mathematical formalism can we accurately define what we mean when we say these things.

To that end, consider an inviscid, homogeneous fluid which is incompressible and bounded below by a rigid horizontal bed of infinite lateral extent. The non-dimensionalised governing equations of irrotational two-dimensional motion of such a fluid, bounded above by a free surface of constant pressure [6], are ([2], p. 14),

$$\begin{cases} \Phi_{zz} + \delta^2 \Phi_{\xi\xi} = 0, \\ \Phi_z = 0, \end{cases} \tag{1}$$

on $z = 0$ (the bottom surface) and,

$$\begin{cases} N + \Phi_\tau + \dfrac{1}{2}\alpha\left(\dfrac{1}{\delta^2}\Phi_z^2 + \Phi_\xi^2\right) = 0, \\ \Phi_z = \delta^2(N_\tau + \alpha\Phi_\xi N_\xi), \end{cases} \tag{2}$$

on $z = 1 + \alpha N$ (the fluid surface), where Φ is the scalar velocity potential, $z = N(\xi, \tau)$ is the wave amplitude at a position ξ and time τ (see Fig. 1, although note it uses convenient scalings which are later introduced), and subscripts denote partial derivatives. Note the parameters which have emerged from the non-dimensionalisation process, $\alpha = \bar{\eta}/h$ and $\delta = h/L$, where the non-dimensionalisation has been done with respect to $\bar{\eta}$ (a characteristic amplitude), h (the undisturbed fluid depth), L (a characteristic wavelength), and a characteristic speed $c_0 = \sqrt{gh}$. These parameters are precisely those needed to rigorously characterise waves: we define a small amplitude wave to correspond to an asymptotic expansion of Eq. 1 and 2 as $\alpha \to 0$, while we define a long wavelength wave to an asymptotic expansion as $\delta \to 0$.

We seek to model small-amplitude long waves (or, equivalently, small-amplitude waves in shallow water),

* Supervisory role: luis.leon@ucd.ie

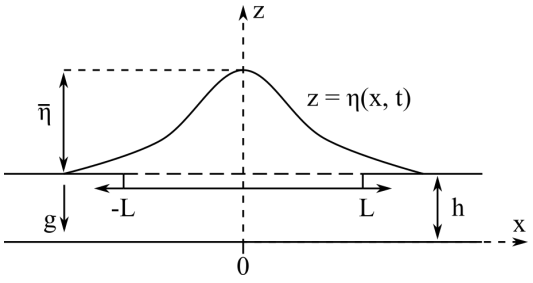

FIG. 1. The scales with respect to which the governing equations Eq. 1 and 2 is nondimensionalised; h is the undisturbed water depth, L is a characteristic wavelength, $\bar{\eta}$ is a characteristic amplitude, and g is acceleration due to gravity.

and so it makes good sense to seek a solution in the limits $\alpha \to 0$ and $\delta \to 0$. As it turns out, a suitable choice of parameters which yields the KdV equation is $\delta^2 = O(\alpha)$ as $\alpha \to 0$ (this is, for example, the method taken by Benetti et al. [7]). This appears, however, altogether unsatisfactory; this arrangement of parameters is so specific that it appears solitary waves should be quite rare in the real world, but we instead find that they arise often and easily in a wide variety of situations.

Let's then return to the problem and instead introduce a scaling for arbitrary δ,

$$x = \frac{\sqrt{\alpha}}{\delta}(\xi - t), \quad t = \frac{\alpha^{\frac{3}{2}}}{\delta}\tau, \quad \phi = \frac{\sqrt{\alpha}}{\delta}\Phi \tag{3}$$

such that Eq. 1 becomes,

$$\begin{cases} \phi_{zz} + \alpha\phi_{xx} = 0, \\ \phi_z = 0 \end{cases} \tag{4}$$

on $z = 0$ and Eq. 2 becomes,

$$\begin{cases} \eta - \phi_x + \alpha\phi_t + \dfrac{1}{2}\left(\phi_z^2 + \alpha\phi_x^2\right) = 0, \\ \phi_z = \alpha\left(-\eta_x + \alpha\eta_t + \alpha\phi_x\eta_x\right) \end{cases} \tag{5}$$

on $z = 1 + \alpha\eta$. Notice that we have essentially replaced δ^2 in these equations by α. The variable scaling of Eq. 3 imposes that our scaled governing equations hold in a frame of reference which is moving in the positive x direction (note the $(x - t)$ term), and for long times τ if $t = O(1)$ (and t is indeed restricted to the order of unity if, as we saw earlier, $\delta^2 = O(\alpha)$). So, then, these scaled variables describe a particular neighbourhood of space and time where we hope solitary waves will always arise so long as we have small amplitude waves, $\alpha \to 0$. Luckily for us, they do arise, and an asymptotic expansion of Eq. 1 and 2 yields the KdV equation ([2], p. 17),

$$2\eta_t + 3\eta\eta_x + \frac{1}{3}\eta_{xxx} = 0, \tag{6}$$

which describes the leading-order contribution to the wave. The exact one-soliton solution to Eq. 6 can be obtained analytically by the inverse transform scattering method (see [8], or [9] for a more modern treatment), in which the water wave problem is reduced to computing the eigenstates of the Schrödinger equation within a finite potential well corresponding to the sign inverse of the initial disturbance from which the solitons evolve [7]. As it happens, there exists a one-to-one correspondence between the number of bound states in the well, and the number of solitons asymptotically evolving from the disturbance, as well as a correspondence between the scattering states and the radiative tail which follows the solitons (see Fig. 5). This procedure yields the one-soliton analytical solution,

$$\eta(x,t) = \eta_0 \,\mathrm{sech}^2[(x - ct)/L] \tag{7}$$

where η_0 is the maximum amplitude of a wave with speed c and characteristic length L,

$$c = c_0\left(1 + \frac{\eta_0}{2h}\right), \tag{8}$$

$$L = \sqrt{\frac{4h^3}{3\eta_0}}. \tag{9}$$

This single solitary wave is significantly different from the sine-like train of waves which we recover in the linear water wave case (see Fig. 3).

As our earlier discussion of a correspondence between bound states of the Schrödinger and the number N of solitons produced by the KdV equation would imply, we can calculate N by considering the size and shape of the initial disturbance (the finite potential well). As noted by Bettini et al. [7], for rectangular initial disturbances of length b and height a,

$$N = 1 + \left[\left(\frac{S}{\pi}\right)\right], \quad S = \frac{b}{h}\sqrt{\frac{3a}{2h}} \tag{10}$$

where [square brackets] denote "integer part of."

Now that the mathematical foundation has been laid, we provide a qualitative description of how solitary waves

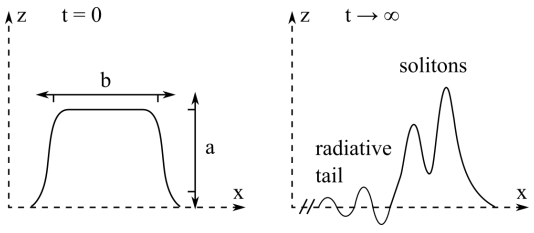

FIG. 2. An initial rectangular disturbance evolves over time into a set of solitons followed by a dampened sine-like wave. Note that the solitons are yet to completely separate from one another.

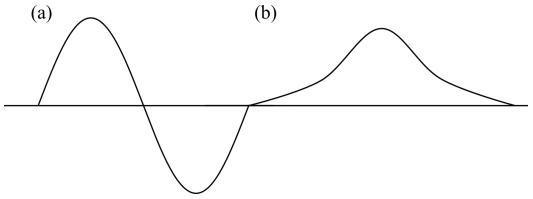

FIG. 3. (a) A sin-like train of waves, as in the solution to the linear water wave problem. (b) The bell-shaped soliton solution to the nonlinear water wave problem.

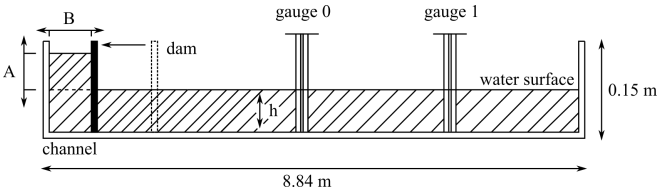

FIG. 4. Apparatus used in the experimental procedure outlined in Sec. IV. Carefully note that while A and B are the dimensions of the initial rectangular wave, the actual rectangular wave produced by extraction of the dam has dimensions $b = 2B$ and $a = A/2$ since the closest end of the tank acts as a reflection plane.

form with the properties that we observe. Looking at the KdV equation (Eq. 6) we see that it consists of three terms: the η_t term describes the time evolution of the wave propagating in one direction; the $\eta\eta_x$ term is clearly nonlinear, and accordingly describes the nonlinear steepening of the wave; the η_{xxx} term describes the linear dispersion of the wave [10]. At a cursory reading, it seems strange that a nonlinear, dispersive partial differential equation could have solutions which neither disperse nor steepen. Indeed, if we have only one of these terms without the other, there can be no solitary solutions. If dispersion alone is present, wave components of different frequencies move at different speeds, with the wave shape changing drastically over time. If nonlinear terms are present alone, the wave steepens due to the continual addition of higher frequencies, until the wave breaks within finite time. We can then qualitatively see that solitary solutions emerge only as a balance between the effects of nonlinearity and dispersion.

We have thus far not discussed the solitons defined in Sec. I, and indeed their nature was not known until computational research (by Zabusky and Kruskal [4]) 70 years after the discovery of the KdV equation. Simulations of the collision of two KdV solitary wave solutions found that rather than destroying one another (as we'd expect in a nonlinear collision), such waves emerged from the collision completely unchanged, save for a phase shift. If we did not have this phase shift, we would not be amiss to say that the waves follow the linear superposition principle (although they of course do not, being nonlinear nonetheless).

III. EXPERIMENTAL PROGRAM

We summarise here the primary predictions and consequences of the theory of solitons described by the KdV equation, Eq. 6. We seek to probe these predictions in the case of long waves in shallow water, for which the KdV equation was originally derived, and demonstrate that as well as being solitary, the waves are solitons.

1. At least one solitary wave evolves from a sufficiently localised initial disturbance, whose shape is described by the one-soliton solution (Eq. 7) to the KdV equation (Eq. 6).

2. The shape of this wave is time-invariant when stable.

3. The number of solitary waves evolving from an initial rectangular wave is given by Eq. 10.

4. The solitary waves are followed by a tail which exhibits dispersion.

5. The speed of the solitary waves depends linearly on their amplitude, as predicted by Eq. 8.

6. The solitary waves are stable under collisions with one another (ie. they are solitons). Note that the KdV equation predicts this only for waves travelling in the same direction, as the equation differs for right-travelling and left-travelling waves. Nevertheless it is believed (see [7]) that KdV solitary waves are solitons in both head-on collisions and catch-up collisions. The KdV equation does, however, predict a phase shift, advancing the larger and slowing the smaller. Quantitatively, in the case of two colliding solitons of amplitudes η_1, η_2 (where $\eta_1 > \eta_2$), the respective time shifts Δ_1, Δ_2 are given by,

$$\begin{cases} \Delta_1 = \left(\dfrac{h}{c_1}\right)\left(\dfrac{1}{K_1}\right)\ln\left[\dfrac{K_1+K_2}{K_1-K_2}\right] \\ \Delta_2 = \left(\dfrac{h}{c_2}\right)\left(\dfrac{1}{K_2}\right)\ln\left[\dfrac{K_1+K_2}{K_1-K_2}\right] \end{cases} \tag{11}$$

where

$$K_i = \sqrt{\frac{3\eta_i}{4h}} \tag{12}$$

and c_i is the speed of the soliton given by Eq. 8.

IV. EXPERIMENTAL PROCEDURE

The experimental program of Sec. III was carried out in a wave tank (see Fig. 4) of length 8.84 m, height

0.15 m, and width 0.08m, consisting of metal sheets. Initial rectangular waves were produced by enclosing a known length B of the end of the tank with a duct-tape-bordered styrofoam dam, pushing in water such that a known height A was reached above the equilibrium depth h, read from a ruler drawn on the furthest wall of the tank. As noted by Hammack and Segur [11], the closest end of the tank forms a reflection plane for this wave, which instantaneously after extraction of the dam is approximately rectangular with height $a = A/2$ and length $b = 2B$. In the course of this work, A and B will refer to the dimensions of the water enclosed at the tank end, while a and b will refer to the dimensions of the instantaneously rectangular wave produced immediately after dam extraction.

The height of the waves was measured at two locations along the length of the tank ($d_1 = 1.95 \pm 0.05$ m, $d_2 = 6.61 \pm 0.05$ m) using two AWP-24-2 water height gauges. The AWP-24-2 probes measure capacitance, which changes with water height, before converting it to an analogue voltage and sending it to a National Instruments USB-6008 data aquisition system for analogue-to-digital conversion. The recieved voltage is proportional to the water height in a highly linear manner, which we determined through the calibration procedure (see Appendix. B, Fig. 10) outlined in the AWP-24-2 user's guide [12].

Note that this method of measuring the wave height as a function of time relies on the assumption that the wave profile does not appreciably change during the observation; since a wave, as a general mean approximation, meets and passes a gauge in no more than a second we take this to be reasonable.

V. RESULTS AND DISCUSSION

We compare the 7 theoretical predictions outlined in Sec. III to the data collected over 38 runs (producing a total of 67 observed solitary waves; see Table. I) of the experimental procedure developed in Sec. IV.

(1) From 38 localized initial disturbances, 67 solitary waves evolved; Fig. 8 depicts 8 of these solitons, produced in 4 runs, with the Eq. 7 prediction overlayed. In general, the agreement is rather good. Note that the agreement seems to decrease past a certain minimum wave height, which becomes most obvious when multiple solitons are produced by a single initial disturbance, each of increasingly smaller amplitude (such as in the top-left figure). We theorize this to not be indicative of a failure of the KdV prediction, but a product of interaction with noise (other non-solitary waves produced by dam extraction), which becomes non-negligible at small wave heights and warps the solitary wave. Indeed, note the bottom-right figure, in which a small amplitude wave at gauge 0 has very good agreement in an environment with low noise.

(2) The solitary waves produced are, rather expectedly, not entirely time-invariant in form - energy is, of course, lost to internal friction and other dissipative forces, reducing the amplitude over time. Qualitatively, looking at Fig. 8, it does immediately appear that the shape remains more-or-less constant between gauges, with only an amplitude decrease.

(3) In Table. I we compare the number (N_{pred}) of solitons predicted (by Eq. 10) to emerge from a given set of initial wave parameters (height A and length B) to the actual observed number, N_{obs}. In order to quantitatively

TABLE I. Comparison of the number (N_{pred}) of solitons predicted by Eq. 10 to emerge from an initial wave of dimensions $A \times B$ m to the actual observed number, N_{obs}. As discussed in Sec. V, the Ursell number U tells us how balanced dispersive and nonlinear effects are; 1 is perfectly balanced, with larger values indicating less balance and so less solitary behaviour.

	A (m)	B (m)	N_{pred}	N_{obs}	U
0	0.05	0.20	2	2	18.52
1	0.06	0.10	1	1	05.56
2[b]	0.05	0.15	2	2	10.42
3	0.06	0.15	2	2	12.50
4	0.04	0.20	2	2	14.81
5	0.05	0.20	2	2	18.52
6	0.05	0.15	2	2	10.42
7	0.06	0.15	2	2	12.50
8	0.07	0.15	2	2	14.58
9	0.04	0.30	3	2[a]	33.33
10	0.03	0.30	2	2	25.00
11	0.02	0.30	2	2	16.67
12	0.04	0.25	2	2	23.15
13	0.03	0.25	2	2	17.36
14	0.02	0.25	2	2	11.57
15	0.08	0.10	2	2	07.41
16	0.07	0.10	1	1	06.48
17[c]	0.06	0.10	1	1	05.56
18	0.05	0.10	1	1	04.63
19	0.06	0.15	2	2	12.50
20	0.05	0.15	2	2	10.42
21	0.03	0.15	1	1	06.25
22	0.02	0.15	1	1	04.17
23	0.04	0.15	2	2	08.33
24	0.03	0.20	2	2	11.11
25	0.04	0.20	2	2	14.81
26	0.05	0.20	2	2	18.52
27	0.06	0.20	2	2	22.22
28	0.03	0.25	2	2	17.36
29	0.04	0.25	2	2	23.15
30	0.05	0.25	3	2[a]	28.94
31	0.02	0.40	3	3	29.63
32	0.03	0.12	1	1	04.00
33	0.04	0.12	1	1	05.33
34	0.05	0.12	2	2	06.67
35	0.05	0.15	2	2	10.42
36	0.02	0.15	1	1	04.17
37	0.03	0.15	1	1	06.25

[a] Incorrect prediction.
[b] Fig. 9 (top).
[c] Fig. 9 (middle).

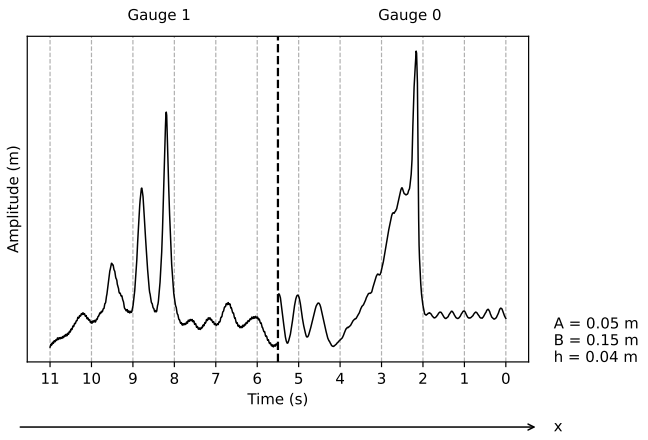

FIG. 5. Time evolution of an initial rectangular wave into three distinct solitons. At gauge 0 the solitons have only just begun to split, while at gauge 1 the solitons have fully split into a rank-ordered set.

understand how accurate we ought to expect Eq. 10 to be for a given wave, we recall from Sec. II that solitary waves form only when dispersive and nonlinear effects are balanced. As it so happens, we have diligently defined (in Sec. II) the parameters α and δ such that this condition is met when

$$U = \frac{\alpha}{\delta} = \frac{b^2 a}{h^3} = O(1), \tag{13}$$

where U is called the Ursell number ([13], §2.9.1, pp. 172-174). While we would expect theoretical solitary waves to be on this precise order of unity, we are under no illusions that any water waves will be so precisely defineable as being solitary. Instead the Ursell number is used in Table. I to explain why observations may not entirely agree with predictions; the higher U is for a given wave, the further we expect it to deviate from the behaviour of a solitary wave.

With this in mind, we note that of a sample of 38 initial disturbances, 36 were predicted correctly (a 95% accuracy rate). Both of the incorrect predictions (for runs 9 and 30) occured for initial wave dimensions with a high Ursell number ($U = 33.33$ and $U = 28.94$ respectively). These were the highest and third-highest U numbers of any waves measured; note that the second-highest U number belongs to run 31, which was correctly predicted and is visible in Fig. 8 (bottom-right). While this is certainly a small sample size of Ursell numbers, it potentially indicates that Ursell number is not a powerful predictor of how solitary we ought to expect a wave to behave.

(4) The following tail is clearly visible in any of the data collected; a particularly clear sample is found in Fig. 6. The tail is, indeed, dispersive in nature, with long wavelengths being fastest.

(5) The actual measured speed c_{sp} of the largest solitary wave of each run in Table. I was calculated simply as the ratio of the distance between the gauges ($d_{12} = 4.66 \pm 0.05$ m) to the time taken to travel that distance, taken as the time between the peak amplitudes in each set of data. The normalized difference of c_{sp} from c_0 (ie. $c_{sp}/c_0 - 1$) was then plotted against the relative average amplitude $\langle \eta_0 \rangle / h$, where $\langle \eta_0 \rangle$ is the average amplitude of the wave between the two gauges.

Referring to Fig. 7, the red line is the prediction for speeds given by Eq. 8, with a slope of $m_p = 0.5$ and a y-intercept of 0. The blue line is an orthogonal distance regression (ODR) fit to the data, with uncertainties included in its parameter calculations. Note that an ODR fit was chosen rather than a conventional ordinary least squares fit due to the advantage that an ODR fit allows for the inclusion of errors on both the independent and dependent variables, which in this case we have (see the errorbars). The computed parameters of the ODR fit give a slope of $m = 0.43 \pm 0.08$ and a y-intercept of -0.016 ± 0.028; the true value does indeed lie within these uncertainties. The theoretical line also lies within the 95% confidence band for the true location of the linear fit line, so we conclude that the speed of the solitary waves does, rather precisely, follow the prediction of Eq. 8.

Note that the linear fit qualitatively looks like the theoretical line simply shifted down the y-axis by a small amount; this aligns with what we should expect, given that the speed of any real wave will almost always be lower than the theoretical prediction, due to the loss of kinetic energy to internal friction and other dissipative forces.

(6) As follows somewhat directly from the above conclusion, the solitary waves do indeed travel in rank-ordered groups, with the faster high-amplitude waves in front. Looking at Fig. 5, note that the wave peaks lay on a straight line, precisely because the relative velocities are proportional to their amplitude as discussed above.

(7) We have thus far established the solitary nature

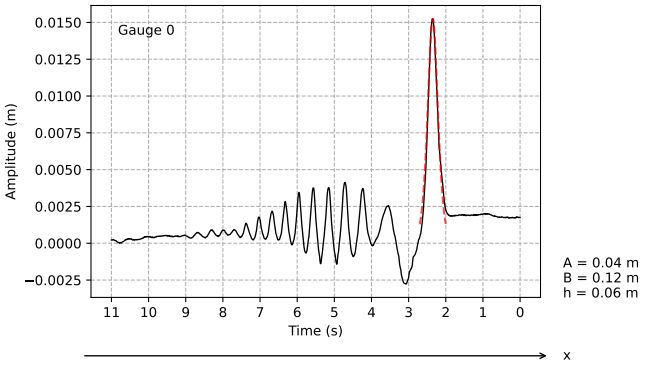

FIG. 6. A soliton approximately 2.5 seconds after creation. Note the dispersive radiative tail which follows directly behind the soliton. Dashed red line is a plot of the soliton predicted by Eq. 7 for the given initial wave parameters.

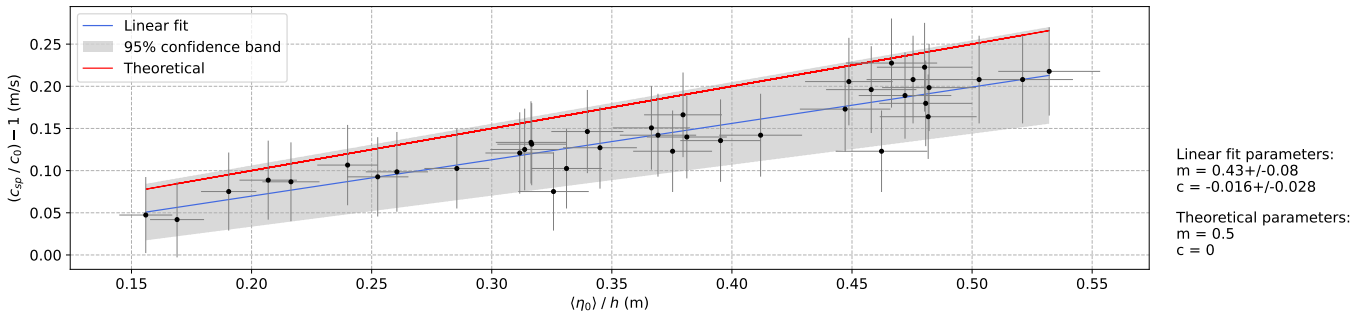

FIG. 7. Relative difference of the measured speed c_{sp} from the characteristic speed c_0 versus the relative average amplitude $\langle\eta_0\rangle /h$ for a sample of 38 solitons (see Table. I) at a water depth $h = 0.06$ m (note that only the largest soliton from each run was taken for simplicity). Theoretical is the prediction of Eq. 8.

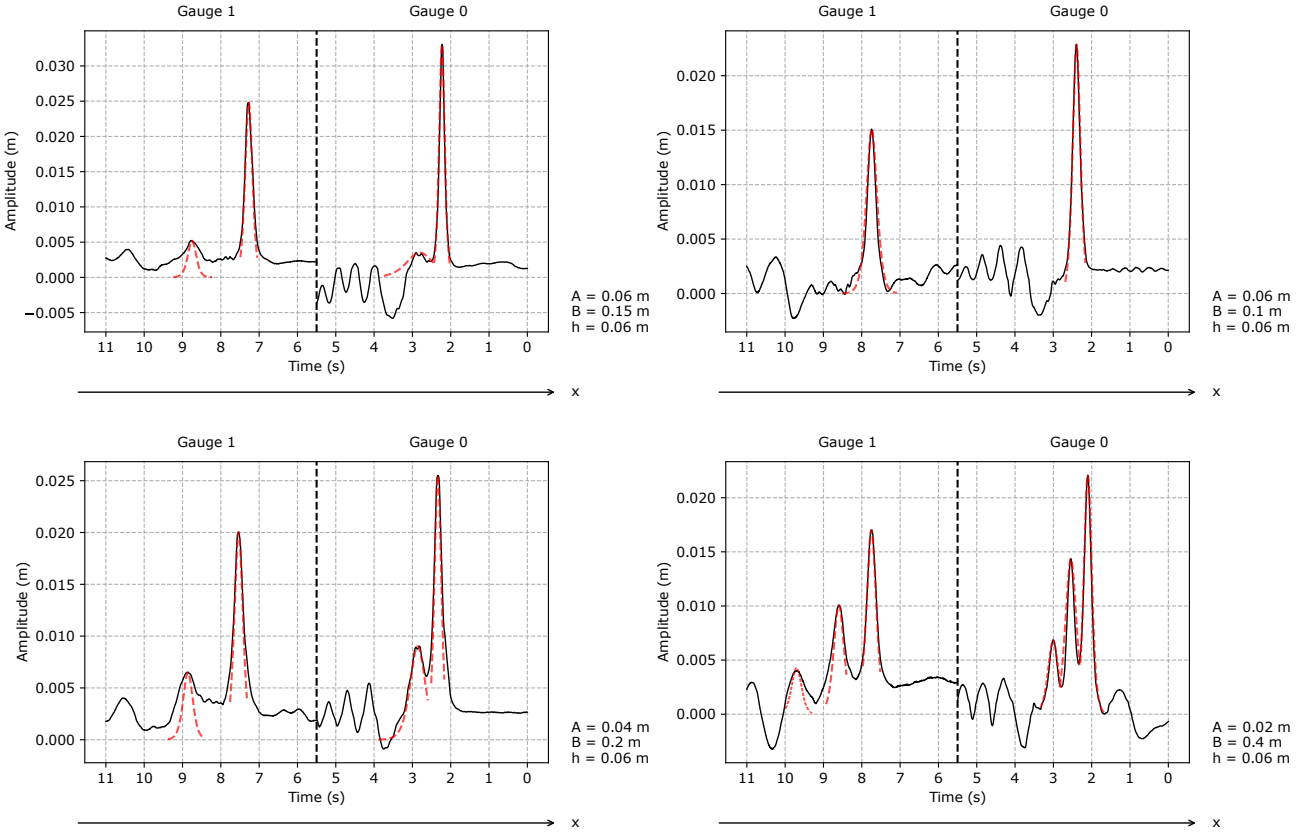

FIG. 8. The one-soliton KdV equation solution (Eq. 7 plotted (red) on top of each of the 8 solitons produced in 4 runs of the experiment, with η_0 adjusted to the observed max amplitude of the soliton. Waves reach gauge 0 first before continuing towards gauge 1; waves at each gauge are moving left to right.

of the waves produced; we now seek to establish their soliton nature in catch-up collisions. Following the same procedure illustrated in Fig. 4 and described in Sec. IV, one initial disturbance $\mathbb{B}$ ($A = 0.06$ m, $B = 0.1$ m, $h = 0.06$ m) was launched, almost immediately followed by another initial disturbance $\mathbb{A}$ ($A = 0.05$ m, $B = 0.15$ m, $h = 0.06$ m), where the maximum amplitude $\eta_A > \eta_B$. This was done using two similar dams, enclosing a total length $B_A + B_B$ at the end of the tank; the dam associated with disturbance $\mathbb{B}$ was extracted, followed in quick succession by the dam of disturbance $\mathbb{A}$.

Then, the waves $\mathbb{A}$ and $\mathbb{B}$ were reproduced alone, without collision, with the apparatus set up exactly as if two solitons were to be launched. Recording $\mathbb{A}$ was treated as a standard measurement, since in the two-disturbance run $\mathbb{A}$ is launched from the furthest end as usual; disturbance $\mathbb{B}$, in the two-disturbance run, was launched from a position shifted down the tank to accomodate the disturbance $\mathbb{A}$ set up behind it. As such, in the one-disturbance run for $\mathbb{B}$ the apparatus was set up as if two waves were

to be launched, but only $\mathbb{B}$ was actually launched.

The result is recorded in Fig. 9; clearly the larger amplitude $\mathbb{A}$ overtook $\mathbb{B}$ between gauge 0 and 1 (bottom), and comparing to the records of $\mathbb{A}$ and $\mathbb{B}$ evolving without collision it qualitatively appears that the shape is largely conserved through the collision. We do note that $\mathbb{A}$ is somewhat wider in the collision case, but this is likely caused by noise as procuring the data for collisions proved difficult with the available dam system and wave tank length.

Shifting the plot of wave $\mathbb{B}$ evolving alone (Fig. 9 (middle)) such that the peak location in time matches that of wave $\mathbb{A}$ evolving alone (Fig. 9 (top)), we ascertain the phase shifts caused by collision, as predicted by the KdV equation. Using the fine time scale in Fig. 9, we observe phase shifts of $\Delta_A = 0.61 \pm 0.01$ s and $\Delta_B = 0.64 \pm 0.01$ s for waves $\mathbb{A}$ and $\mathbb{B}$ in a catch-up collision with one another; the theoretical values predicted by Eq. 11 are $\Delta_A = 0.57 \pm 0.05$ s and $\Delta_B = 0.61 \pm 0.06$ s; these theoretical and observed values are in good agreement, being within the uncertainty on both.

VI. CONCLUSION

We have shown that 7 primary predictions of the solitary solution to the KdV equation (the evolution of solitary waves from a localised disturbance, the time-invariance of said wave's shapes, the predictability of the number of waves which evolve given certain initial dimensions, the dispersive tail which follows them, the linear relationship between amplitude and speed, and the stability under collisions with one another) hold for nonlinear waves in water, provided their Ursell number is sufficiently low (ie. dispersive and nonlinear effects balance one another out). We make this conclusion by drawing emphasis to the following facts established by our data:

- the relationship between amplitude and speed is given by a straight line with parameters $m = 0.43 \pm 0.08$, $c = -0.016 \pm 0.028$, well within the theoretical KdV-derived parameters of $m = 0.5$, $c = 0$.
- the number of solitons produced by an initial wave disturbance is well-predicted by the KdV-derived value (a 95% accuracy, with incorrect predictions occuring only for high Ursell numbers).
- the phase shift predicted by the KdV equation in a catch-up collision (in the example analysed in this work, $\Delta_A = 0.57 \pm 0.05$ s, $\Delta_B = 0.61 \pm 0.06$ s) accurately models observed values ($\Delta_A = 0.61 \pm 0.01$ s, $\Delta_B = 0.64 \pm 0.01$ s) within the stated uncertainties.

Appendix A: Code

This appendix will consist of key snippets of code, with comments removed where they would be overly verbose and array definitions/code/functions shortened or skipped (noted by ellipses) when considered unneccesary to quickly understanding the structure of the code; the full unedited code is available at the Git repository here (all raw data is also available). The National Instruments USB-6008 was communicated with through a custom Python helper class,

```
from pydaqmx_helper.adc import ADC

# Initialise ADC class instance.
myADC = ADC()

# There are two wave height gauges which we
# assign to channel 0 and 1, where 0 is the
# gauge closest to the wave generation
# point (the left-most side of the tank).
myADC.addChannels([0, 1],
minRange = -4.5,
maxRange = 4.5)
```

and data collection and analysing was handled by a standard set of packages,

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np
import scipy.signal
from scipy import odr
import mpmath
import uncertainties as unc
from uncertainties import ufloat
from uncertainties import unumpy as unp
```

Note the Uncertainties package, which handles all uncertainty propagation throughout the data analysis. Calibration of the wave height gauges was performed as follows, taking a single point of data at multiple known water heights and plotting voltage against known water height,

```
calibration_data = print(myADC.sampleVoltages(1, 100))

# calibration_voltages0 corresponds to the left-most
gauge, etc., and has been manually populated.
calibration_heights = np.array([0.03, ..., 0.1])
calibration_voltages0 = [...]
calibration_voltages1 = [...]

# Define line for use in creating linear fit with scipy
curve_fit().
def line(x, m, c):
return m * x + c

# Generate array of optimum line parameters popt, where
popt[0] = m, popt[1] = c, and parameter covariance matrix
pcov.
# For gauge 0.
popt0, pcov0 = scipy.optimize.curve_fit(line,
calibration_heights, calibration_voltages0)

# Initialise correlated system of values for propagation
handling by the Uncertainties package.
m0, c0 = unc.correlated_values(popt0, pcov0)

# The same was done for gauge 1.

...
```

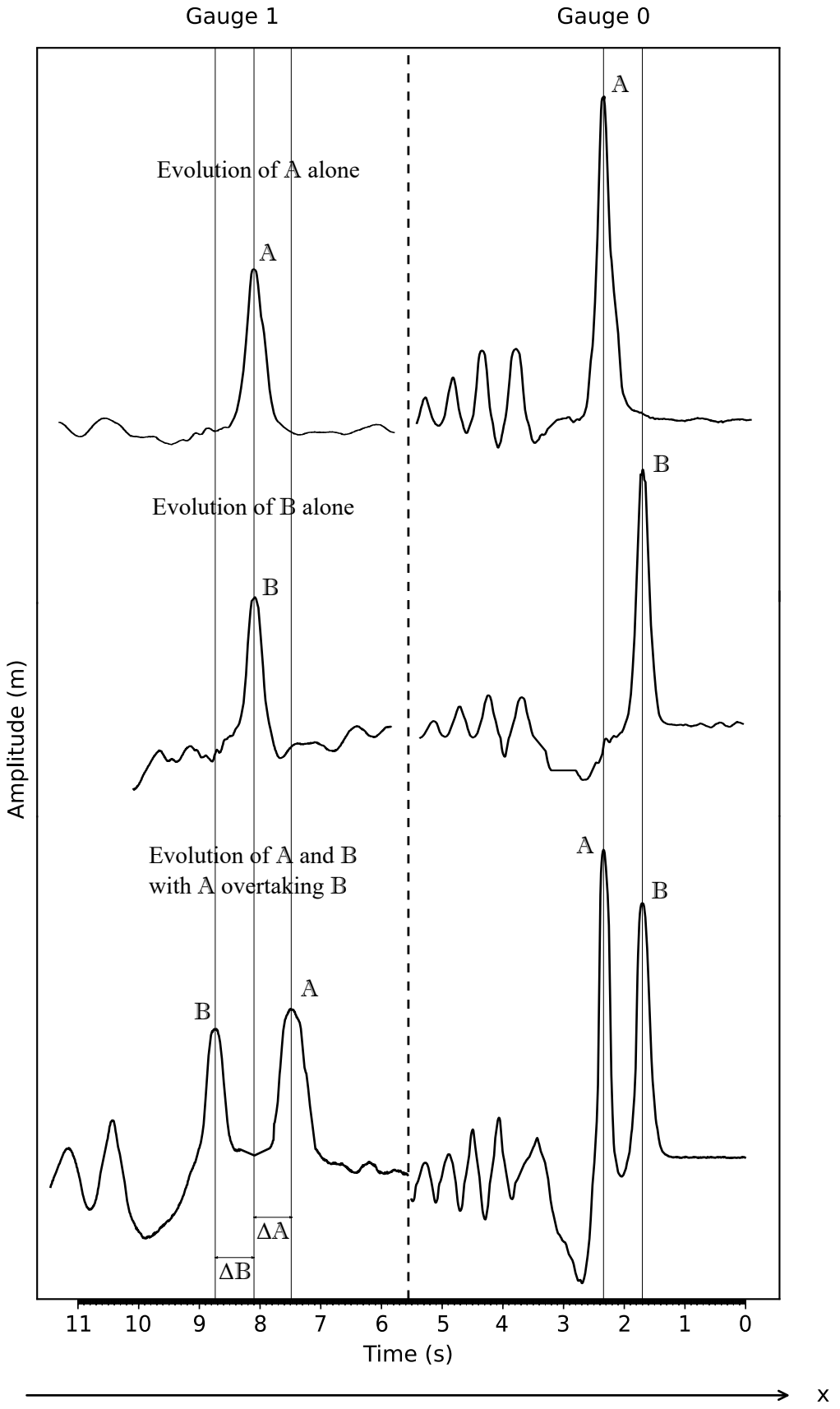

FIG. 9. (Top) Evolution of an initial disturbance $\mathbb{A}$ (run 2 in Table. I) of height $A = 0.05$ m, length $B = 0.15$ m, without collision with another solitary wave. (Middle) Evolution of an initial disturbance $\mathbb{B}$ (run 17 in Table. I) of height $A = 0.06$ m, length $B = 0.1$ m, without collision with another solitary wave. (Bottom) Evolution of both the larger ($\mathbb{A}$) and smaller ($\mathbb{B}$) solitary wave, launched one after the other as described in Sec. III - $\mathbb{A}$ overtakes $\mathbb{B}$ between the gauges.

```
print("Height-voltage constant 0: ", m0)
print("Height-voltage constant 1: ", m1)
```

The plots associated with these linear fits are recorded in Fig. 10. Once calibration is complete, measurements are taken by the following method,

```
def take_height_measurements(run_num, A, B, eq_h, n =
1100, r = 100):
        time_elapsed = n/r

        # Take n samples total of the voltage, taking r
        samples per second.
        raw_data = myADC.sampleVoltages(n, r)

        # Initialise an empty nparray for to transfer
        raw_data into for easier processing.
        data = np.empty([0])

        # Transfer data from standard array to numpy
        arrays.
        # data0 corresponds to the first (left-most) gauge
        while data1 corresponds to the right-most gauge.
        data0 = np.append(data, raw_data[0])
        data1 = np.append(data, raw_data[1])

        # Initialise empty np arrays for storing height
        data.
        height_data0 = np.empty([0])
        height_data1 = np.empty([0])

        # Convert voltages to heights in m.
        for v in data0:
        h = get_water_height(v, 0)
        height_data0 = np.append(height_data0, h)

        for v in data1:
        h = get_water_height(v, 1)
        height_data1 = np.append(height_data1, h)

        header = f"Columns are height in m for gauge 0,
        gauge 1. \nInitial wave height A = {A} m, length
        B = {B} m."
        header += f"\nEquilibrium water height h = {eq_h}
        m. \n{n} total samples taken at {r} Hz over
        {time_elapsed} seconds."
        np.savetxt(f"height_data_{run_num}.txt",
        np.column_stack([height_data0, height_data1]),
        header = header)
```

where the `get_water_height()` function is defined by,

```
# Returns the water height from the measured voltage v, at
the specified gauge.
def get_water_height(v, gauge):
        if (gauge == 0):
                return (v-c0)/m0
        elif (gauge == 1):
                return (v-c1)/m1
```

The collected data is then normalized by a `normalize_height_data()` function so that the water height at equilibrium is always 0. The data is then ready for analysis, which consists primarily of running the data through the `wave_speed_comparison()`, `get_number_of_solitons()`, and `plot_theoretical_wave()` functions, which are reproduced here:

```
def wave_speed_comparison(h = 0.06, runs = range(0, 38),
best_fit = False, plot_errors = False):
        # Calculating c_0 with uncertainty on h.
        c_0 = get_c_0(9.8, ufloat(h, 0.005))

        measured_speeds = np.array([])
        avg_peak_heights = np.array([])

        for i in runs:
                c_sp, peak_height, u_peak_height =
                get_measured_speed(i)
                nom_n_0 = peak_height[0]
                u_n_0 = u_peak_height[0]
                nom_n_1 = peak_height[1]
                u_n_1 = u_peak_height[1]

        n_0 = ufloat(nom_n_0, u_n_0)
        n_1 = ufloat(nom_n_1, u_n_1)

        avg_n = (n_0 + n_1)/2

        avg_peak_heights = np.append(avg_n,
        avg_peak_heights)
        measured_speeds = np.append(c_sp,
        measured_speeds)

        #print("Average peak heights: ", avg_peak_heights)
        #print("Measured speeds: ", measured_speeds)
        #print("Theoretical speeds: ", wave_speed(c_0, h,
        avg_peak_heights))

        x = unp.nominal_values(avg_peak_heights/h)
        xerr = unp.std_devs(avg_peak_heights/h)
        y = unp.nominal_values((measured_speeds/c_0)-1)
        yerr = unp.std_devs((measured_speeds/c_0)-1)

        # Plot best fit using an ODR method - this method
        is chosen due to the ability to generate
        parameters when there is
        # uncertainty on both the x and y axis variables.
        if(best_fit == True):
        data = odr.RealData(x, y, xerr, yerr)

        my_odr = odr.ODR(data, odr.unilinear)
        out = my_odr.run()

        # Output computed parameters, where popt[0] = m,
        popt[1] = c, covariance matrix pcov, and standard
        error perr.
        popt = out.beta
        pcov = out.cov_beta
        perr = out.sd_beta

        # Initialise correlated system of values for
        propagation handling by the Uncertainties package.
        m, c = unc.correlated_values(popt, pcov)

        x_fit = np.linspace(min(x), max(x), 1000)
        y_fit = line(x_fit, m, c)
        nom_y = unp.nominal_values(y_fit)

        # Plot linear fit line.
        plt.plot(x_fit, nom_y, color = "royalblue",
        linewidth = 1)

        # Plot 95% confidence band, making sure to include
        covariance since fit parameters are obviously
        correlated.
        # 95% confidence band parameters.
        popt_upper_bound = popt + 2 * perr
```

```
        popt_lower_bound = popt - 2 * perr

        fit_upper = line(x_fit, popt_upper_bound[0],
        popt_upper_bound[1])
        fit_lower = line(x_fit, popt_lower_bound[0],
        popt_lower_bound[1])
        plt.fill_between(x_fit, fit_lower, fit_upper,
        color = "black", alpha = 0.15, linewidth = 0)

        print(f"Linear fit line slope m = {m}.")

        # Plot theoretical line.
        plt.plot(x, unp.nominal_values(((wave_speed(c_0,
        h, avg_peak_heights))/c_0)-1), color = "red",
        linewidth = 1)

        print("Theoretical slope m = 0.5.")

        # Scatter data points.
        plt.scatter(x, y, c = "black", s = 10, zorder =
        5)

        # Plot errorbars if prompted to.
        if (plot_errors == True):
                plt.errorbar(x, y, yerr, xerr, fmt =
                "none", ecolor = "gray", elinewidth =
                0.6)

        text1 = f"Linear fit parameters:\nm = {m}\nc =
        {c}"
        text2 = f"Theoretical parameters:\nm = {0.5}\nc =
        {0}"
        plt.text(0.585, 0.075, text1)
        plt.text(0.585, 0, text2)
        plt.legend(["Linear fit", "95% confidence band",
        "Theoretical"])
        plt.xlabel(r"$\left\langle\eta_0\right\rangle\; /
        \;h$ (m)")
        plt.ylabel(r"$(c_{sp}\; / \;c_0) - 1$ (m/s)")
        plt.grid(linestyle = "--")

        plt.savefig("Wave speed comparison.pdf",
        bbox_inches = "tight")


def get_number_of_solitons(run_num):
        # Get the height A and length B of the initial
        wave from the data file.
        data_file =
        open(f"norm_height_data_{run_num}.txt")
        for line in data_file:
                if "A" in line:
                        initial_wave_params_line = line
                elif "h" in line:
                        water_equilibrium_height_line =
                        line

        # Split the initial_wave_params_line into
        individual words. Then, try and convert these
        words into floats. If this is not possible, pass.
        If it is, add the number to params.
        params = np.array([])
        for word in initial_wave_params_line.split():
                try:
                        word = float(word)
                        params = np.append(word, params)
                except:
                        pass

        A = params[1]
        B = params[0]

        a = A/2
        b = 2*B

        # By the same method as above, look for any
        numbers in the words which constitute the
        water_equilibrium_height_line.
        # Once the single height value has been found,
        break from the loop as the other words don't need
        to be checked.
        for word in
        water_equilibrium_height_line.split():
                try:
                        h = float(word)
                break
                except:
                        pass

        return number_of_solitons(a, h, b)

def plot_theoretical_wave(run_num, t, t_1, gauge = None,
sol_0_offset = 0, sol_1_offset = 0, plot_together =
False):
        nom_height_data0, u_height_data0,
        nom_height_data1, u_height_data1 =
        np.loadtxt(f"norm_height_data_{run_num}.txt",
        unpack = "true")

        # Get the equilibrium water height from the data
        file.
        data_file =
        open(f"norm_height_data_{run_num}.txt")
        for line in data_file:
                if "A" in line:
                        initial_wave_params_line = line
                if "h" in line:
                        water_equilibrium_height_line =
                        line
                if "seconds" in line:
                        time_elapsed_line = line

        # Look for any numbers in the words which
        constitute the water_equilibrium_height_line.
        # Once the single height value has been found,
        break from the loop as the other words don't need
        to be checked.
        for word in
        water_equilibrium_height_line.split():
                try:
                        h = float(word)
                        break
                except:
                        pass

        # Split the initial_wave_params_line into
        individual words. Then, try and convert these
        words into float. If this is not
        # possible, pass. If it is, add the number to
        params.
        params = np.array([])
        for word in initial_wave_params_line.split():
                try:
                        word = float(word)
                        params = np.append(word, params)
                except:
                        pass

        A = params[1]
        B = params[0]
```

```
# Split the time_elapsed_line into individual
words. Then, try and convert these words into
float. If this is not
# possible, pass. If it is, add the number to
sampling_data.
sampling_data = np.array([])
for word in time_elapsed_line.split():
        try:
                word = float(word)
                sampling_data = np.append(word,
                sampling_data)
        except:
                pass

time_elapsed = sampling_data[0]

# Get highest two peaks from data, where n_0 is
the highest - these should correspond to the
soliton peaks.
peak_times_g_0, properties_g_0 =
scipy.signal.find_peaks(nom_height_data0, height
= [-0.06, 0.06])
peak_heights_g_0 =
np.sort(properties_g_0["peak_heights"])
n0_g_0 = np.abs(peak_heights_g_0[-1])
n1_g_0 = np.abs(peak_heights_g_0[-2])

peak_times_g_1, properties_g_1 =
scipy.signal.find_peaks(nom_height_data1, height
= [-0.06, 0.06])
peak_heights_g_1 =
np.sort(properties_g_1["peak_heights"])
n0_g_1 = np.abs(peak_heights_g_1[-1])
n1_g_1 = np.abs(peak_heights_g_1[-2])

# Defining uncertainty on water depth measurement
h.
uh = ufloat(h, 0.005)

# Calculating c_0 with uncertainty on h.
c_0 = get_c_0(9.8, uh)

# Calculate wave speed c of soliton with peak
n0/n1.
c0_g_0 = wave_speed(c_0, uh, n0_g_0)
nom_c0_g_0 = unp.nominal_values(c0_g_0)

...

c1_g_1 = wave_speed(c_0, uh, n1_g_1)
nom_c1_g_1 = unp.nominal_values(c1_g_1)

# Calculate characteristic wave length in x, L_x,
and characteristic wave length in t, L_t, for two
solitons.
L_x0_g_0 = char_length(uh, n0_g_0)
nom_L_x0_g_0 = unp.nominal_values(L_x0_g_0)

...

L_t1_g_1 = L_x0_g_1/c0_g_1
nom_L_t1_g_1 = unp.nominal_values(L_t1_g_1)

d_1 = ufloat(1.945, 0.05)
d_2 = ufloat(6.61, 0.05)

dt_10_g_0 = d_1/c0_g_0 + sol_0_offset
nom_dt_10_g_0 = unp.nominal_values(dt_10_g_0)

...

dt_21_g_1 = d_2/c1_g_1 + sol_1_offset
nom_dt_21_g_1 = unp.nominal_values(dt_21_g_1)

soliton_num = get_number_of_solitons(run_num)

if (gauge == 0 and plot_together == False):
        if (soliton_num >= 2):
                plt.plot(t_1,
                kdv_solitary_solution(n1_g_0,
                nom_dt_11_g_0, t_1, nom_c1_g_0,
                nom_L_t1_g_0), "--", color =
                "red", zorder = 5, alpha = 0.7)

# Plot the KdV solution.
plt.plot(t, kdv_solitary_solution(n0_g_0,
nom_dt_10_g_0, t, nom_c0_g_0, nom_L_t0_g_0),
"--", color = "red", zorder = 5, alpha = 0.7)

# Plot the height data for gauge 0.
plt.plot(np.linspace(0, time_elapsed,
np.size(nom_height_data0)), nom_height_data0,
color = "black", linewidth = 1)

plt.xlabel("Time (s)")
plt.ylabel("Amplitude (m)")
plt.grid(linestyle = "--")

ax = plt.gca()
ax.invert_xaxis()

plt.text(1.05, 0, f"A = {A} m\nB = {B} m\nh = {h}
m", transform = ax.transAxes)
plt.text(0.06, 0.9, "Gauge 0", transform =
ax.transAxes)

# Add an arrow indicating positive x direction.
ax.annotate("",
xy=(1.02, -0.2),
xycoords="axes fraction",
xytext=(-0.02, -0.2),
arrowprops=dict(arrowstyle="->", color="black"))
plt.text(1.05, -0.213, "x", transform =
ax.transAxes)

plt.savefig(f"Run {run_num} Gauge 0 Plot.pdf",
bbox_inches = "tight")
plt.show()

elif (gauge == 1 and plot_together == False):
        if (soliton_num >= 2):
                plt.plot(t_1,
                kdv_solitary_solution(n1_g_1,
                nom_dt_21_g_1, t_1, nom_c1_g_1,
                nom_L_t1_g_1), "--", color =
                "red", zorder = 5, alpha = 0.7)

# Plot the KdV solution.
plt.plot(t, kdv_solitary_solution(n0_g_1,
nom_dt_20_g_1, t, nom_c0_g_1, nom_L_t0_g_1),
"--", color = "red", zorder = 5, alpha = 0.7)

# Plot the height data from gauge 1.
plt.plot(np.linspace(0, time_elapsed,
np.size(nom_height_data1)), nom_height_data1,
color = "black", linewidth = 1)

plt.xlabel("Time (s)")
plt.ylabel("Amplitude (m)")
```

```
plt.grid(linestyle = "--")

ax = plt.gca()
ax.invert_xaxis()

plt.text(1.05, 0, f"A = {A} m\nB = {B} m\nh = {h}
m", transform = ax.transAxes)
plt.text(0.83, 0.9, "Gauge 1", transform =
ax.transAxes)

# Add an arrow indicating positive x direction.
ax.annotate("",
xy=(1.02, -0.2),
xycoords='axes fraction',
xytext=(-0.02, -0.2),
arrowprops=dict(arrowstyle="->", color='black'))
plt.text(1.05, -0.213, "x", transform =
ax.transAxes)

plt.savefig(f"Run {run_num} Gauge 1 Plot.pdf",
bbox_inches = "tight")
plt.show()

elif (plot_together == True):
        if (soliton_num >= 2):
                plt.plot(t_1,
                kdv_solitary_solution(n1_g_0,
                nom_dt_11_g_0, t_1, nom_c1_g_0,
                nom_L_t1_g_0), "--", color =
                "red", zorder = 5, alpha = 0.7)
plt.plot(t_1, kdv_solitary_solution(n1_g_1,
nom_dt_21_g_1, t_1, nom_c1_g_1, nom_L_t1_g_1),
"--", color = "red", zorder = 5, alpha = 0.7)

# Plot KdV equation solution for gauge 0, and then
gauge 1.
plt.plot(t, kdv_solitary_solution(n0_g_0,
nom_dt_10_g_0, t, nom_c0_g_0, nom_L_t0_g_0),
"--", color = "red", zorder = 5, alpha = 0.7)
plt.plot(t, kdv_solitary_solution(n0_g_1,
nom_dt_20_g_1, t, nom_c0_g_1, nom_L_t0_g_1),
"--", color = "red", zorder = 5, alpha = 0.7)

# Plot the first half of the wave height data for
the first gauge, then plot the last half of the
wave height
# data for the second gauge.
plt.plot(np.linspace(0, time_elapsed/2,
len(nom_height_data0[:len(nom_height_data0)//2])),
nom_height_data0[:len(nom_height_data0)//2],
color = "black", linewidth = 1)
plt.plot(np.linspace(time_elapsed/2,
time_elapsed,
len(nom_height_data1[len(nom_height_data1)//2:])),
nom_height_data1[len(nom_height_data1)//2:],
color = "black", linewidth = 1)

plt.axvline(x = time_elapsed/2, linestyle = "--",
color = "black")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (m)")
plt.grid(linestyle = "--")

ax = plt.gca()
ax.invert_xaxis()

plt.text(1.05, 0, f"A = {A} m\nB = {B} m\nh = {h}
m", transform = ax.transAxes)
plt.text(0.7, 1.05, "Gauge 0", transform =
ax.transAxes)
plt.text(0.2, 1.05, "Gauge 1", transform =
ax.transAxes)

# Add an arrow indicating positive x direction.
ax.annotate("",
xy=(1.02, -0.2),
xycoords='axes fraction',
xytext=(-0.02, -0.2),
arrowprops=dict(arrowstyle="->", color='black'))
plt.text(1.05, -0.213, "x", transform =
ax.transAxes)

plt.savefig(f"Run {run_num} Both Gauges
Plot.pdf", bbox_inches = "tight")
plt.show()

# Print header from data file.
data_file =
open(f"norm_height_data_{run_num}.txt")
for line in data_file:
        if "#" in line:
                print(line)

time_shifts = time_shift((c0_g_0+c0_g_1)/2,
(c1_g_0+c1_g_1)/2, (n0_g_0+n0_g_1)/2,
(n1_g_0+n1_g_1)/2, uh)
print(f"Time shifts: {time_shifts}")
```

Appendix B: Calibration

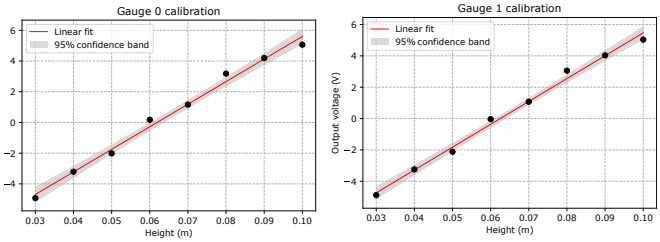

FIG. 10. Calibration curves of the two AWP-24-2 wave height gauges used. For gauge 0, the fit parameters (m, c) which relate voltage and height are $(m_0 = 147 \pm 6, c_0 = -9.1 \pm 0.4)$; for gauge 1, the fit parameters are $(m_1 = 146 \pm 5, c_1 = -9.11 \pm 0.35)$.

[1] J. Russell, *Report on Waves: Made to the Meetings of the British Association in 1842-43* (Harvard University, 1845).

[2] P. Drazin, R. Johnson, and D. Crighton, *Solitons: An Introduction*, Cambridge Texts in Applied Mathematics (Cambridge University Press, 1989).

[3] E. Fermi, P. Pasta, S. Ulam, and M. Tsingou, Studies of Nonlinear Problems, Los Alamos National Lab

10.2172/4376203 (1955).
[4] N. J. Zabusky and M. D. Kruskal, Interaction of "solitons" in a collisionless plasma and the recurrence of initial states, Phys. Rev. Lett. **15**, 240 (1965).
[5] R. S. Johnson, *A Modern Introduction to the Mathematical Theory of Water Waves* (Cambridge University Press, 1997).
[6] Note that this amounts to neglecting surface tension effects. As calculated by Bettini, Minelli, and Pascoli [7], for water at a depth of 6 cm (as used in this work), the error amounts to only a 0.6% correction term to the dispersion relation of the waves, which we consider negligible.
[7] A. Bettini, T. Minelli, and D. Pascoli, Solitons in undergraduate laboratory, American Journal of Physics **51**, 977 (1983).
[8] C. S. Gardner, J. M. Greene, M. D. Kruskal, and R. M. Miura, Korteweg-DeVries Equation and Generalizations. VI. Methods for Exact Solution, Communications on Pure and Applied Mathematics **27**, 97 (1974).
[9] M. A. Ablowitz and P. A. Clarkson, *Solitons, Nonlinear Evolution Equations and Inverse Scattering*, London Mathematical Society Lecture Note Series (Cambridge University Press, 1991).
[10] A.-M. Wazwaz, The Family of the KdV Equations, in *Partial Differential Equations and Solitary Waves Theory* (Springer, Berlin, Heidelberg, 2009) pp. 503–556.
[11] J. L. Hammack and H. Segur, The Korteweg-de Vries equation and water waves, Journal of Fluid Mechanics **65**, 289–314 (1974).
[12] *AWP-24-2 Wave Height Gauge User's Guide*, Akamina Technologies Inc., Ontario, Canada, 1st ed. (2013).
[13] M. W. Dingemans, *Water Wave Propagation Over Uneven Bottoms*, Vol. 13 (World Scientific, 1997).